

C++ Golden Model

Systolic Array Matrix Multiplication Accelerator

Full Software Golden-Model Specification — Version 1.0

Specification item	Current baseline
Reference model	C++17 software golden model
Mathematical operation	$C = A \times W$
Supported dimensions	$1 \leq M, K, N \leq 16$
Input type	Signed INT8
Product type	Signed INT16
Accumulator / output	Signed INT32
Physical accelerator	4×4 weight-stationary systolic array
Current software scope	Pure matrix multiplication only
Excluded for now	Tiling, folding, timing, AXI, DMA, FPGA behavior

Purpose: This document freezes the current software-reference specification before implementation. The C++ model is the independent mathematical reference that future RTL and UVM verification will compare against.

Document Scope

This specification defines the complete current requirements for the C++ golden model used to verify a future INT8 matrix-multiplication accelerator. It intentionally separates the mathematical reference from the hardware implementation.

Core principle: the golden model answers **WHAT** the result must be; RTL answers **HOW** the hardware produces it.

1. Project Objective

Implement a generic, mathematically correct C++ reference model for:

$$C = A \times W$$

where:

$$\begin{aligned} A &\in \mathbb{Z}^{(M \times K)} \\ W &\in \mathbb{Z}^{(K \times N)} \\ C &\in \mathbb{Z}^{(M \times N)} \end{aligned}$$

The model is intended to be the trusted software reference for later RTL simulation, SystemVerilog verification, and UVM scoreboard checking.

2. Current Scope

The golden model supports every valid matrix shape for which:

$$\begin{aligned} 1 &\leq M \leq 16 \\ 1 &\leq K \leq 16 \\ 1 &\leq N \leq 16 \end{aligned}$$

Because M, K, and N are independently variable, there are:

$$16 \times 16 \times 16 = 4096$$

valid dimension combinations. The scope therefore includes rectangular matrices as well as square matrices.

A	W	Result C	Status
1×1	1×1	1×1	Valid
3×7	7×5	3×5	Valid
4×8	8×4	4×4	Valid
16×16	16×16	16×16	Valid

3. Physical Hardware Target

The eventual accelerator targets a ZedBoard based on a Zynq-7000 XC7Z020 device. The baseline hardware is a 4×4 systolic array containing 16 processing elements (PEs).

```
ZedBoard
■
Zynq-7000 XC7Z020
■
■■■ ARM Cortex-A9
■■■ AXI control/data interfaces (future)
■■■ 4×4 systolic array
■■■ 16 PEs
```

Important: the physical hardware is 4×4, but the C++ golden model is intentionally generic up to 16×16 dimensions. The software reference does not simulate a 4×4 array.

4. Mathematical Operation

For each output element:

$$C[i][j] = \sum_{k=0..K-1} A[i][k] \times W[k][j]$$

Expanded for a particular element:

$$\begin{aligned} C[i][j] &= A[i][0] \times W[0][j] \\ &+ A[i][1] \times W[1][j] \\ &+ \dots \\ &+ A[i][K-1] \times W[K-1][j] \end{aligned}$$

Example: 2x2

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \quad W = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$$

$$\begin{aligned} C[0][0] &= 1 \times 5 + 2 \times 7 = 19 \\ C[0][1] &= 1 \times 6 + 2 \times 8 = 22 \\ C[1][0] &= 3 \times 5 + 4 \times 7 = 43 \\ C[1][1] &= 3 \times 6 + 4 \times 8 = 50 \end{aligned}$$

$$C = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$$

5. Matrix Naming

Matrix	Project meaning	Dimensions	Datatype
A	Input / activation matrix	M×K	signed INT8
W	Weight matrix	K×N	signed INT8
C	Output / result matrix	M×N	signed INT32

The second matrix is named W because the hardware uses weight-stationary dataflow. Mathematically, it is simply the second operand of the matrix multiplication.

6. Datatype Specification

Stage	Type	Range / role
A input	int8_t	−128 to +127
W input	int8_t	−128 to +127
Product	int16_t	INT8 × INT8
Accumulator	int32_t	Sum of K products
C output	int32_t	Final accumulated result

$$\text{INT8} \times \text{INT8} \rightarrow \text{INT16 product} \rightarrow \text{INT32 accumulator} \rightarrow \text{INT32 output}$$

Explicit C++ casts should be used where appropriate so the intended arithmetic widths are clear instead of relying on implicit integer promotions.

7. Accumulator Range

The current maximum reduction length is K=16. The largest-magnitude signed INT8 product is 16384 in magnitude (−128×−128 = +16384; mixed-sign extrema are also safely representable in INT16). A conservative current bound is therefore:

$$16 \times 16384 = 262144$$

This is far below the signed INT32 maximum of 2,147,483,647. INT32 is therefore sufficient for the current supported dimension range.

If K or the arithmetic format is expanded later, the accumulator-width requirement must be re-evaluated.

8. Arithmetic Policy

The baseline model performs ordinary signed integer matrix multiplication.

Included

- Signed INT8 inputs
- Signed multiplication
- INT16 product representation
- INT32 accumulation
- INT32 output
- Exact integer matrix multiplication

Excluded

- Bias
- ReLU or other activation functions
- Quantization
- Rounding
- Saturation / clipping
- Scale factors / zero-points
- Floating-point arithmetic
- Fixed-point scaling

The baseline operation is strictly $\mathbf{C} = \mathbf{A} \times \mathbf{W}$.

9. No Saturation

The model does not saturate or clip the output. There is no conversion of a large result to an INT8 range. Results remain INT32.

```
No: result > MAX → MAX  
No: result < MIN → MIN
```

For the current $K \leq 16$ range, the mathematical result fits safely inside INT32, so saturation is unnecessary.

10. Matrix Storage and Memory Management

Matrices use a single contiguous row-major std::vector as their owning storage.

```
A = [ 1 2 3 ]  
    [ 4 5 6 ]  
  
Memory:  
[1][2][3][4][5][6]
```

Element indexing is:

```
index = row × columns + column
```

This representation provides contiguous memory, predictable indexing, good locality, straightforward copying/moving, and safe automatic lifetime management.

Ownership policy

Use RAII and the Rule of Zero. Matrix ownership should not use raw new/delete or malloc/free.

- `std::vector` as the owning buffer
- No manual `new[]` / `delete[]`
- No `malloc()` / `free()`
- Avoid `vector<>` as the primary storage
- Optional `data()` access may expose a non-owning pointer for later interfaces

11. Matrix Class Requirements

```
template <typename T>
class Matrix
{
private:
    std::size_t rows_;
    std::size_t cols_;
    std::vector<T> data_;

public:
    Matrix(std::size_t rows, std::size_t cols);

    std::size_t rows() const;
    std::size_t cols() const;

    T& operator()(std::size_t row, std::size_t col);
    const T& operator()(std::size_t row,
        std::size_t col) const;

    T* data();
    const T* data() const;
};
```

The exact implementation may add useful helpers such as `fill()`, `print()`, and deterministic random generation, but those helpers must not change the core matrix semantics.

12. Dimension Validation

Zero-sized matrices are invalid for this project. Every matrix must have `rows > 0` and `columns > 0`.

Shape	Status
0×4	Invalid
4×0	Invalid
0×0	Invalid
1×1	Valid
1×16	Valid
16×1	Valid
16×16	Valid

13. Multiplication Validation

For $A=M \times K$ and $W=K \times N$, multiplication is valid only when `A.columns == W.rows`.

```
Valid:
A = 4×8
W = 8×5
C = 4×5
```

```
Invalid:
A = 4×8
W = 7×5
```

A dimension mismatch must be reported as an error; the function must not silently produce a result.

14. Input Value Validation

When data originates in a wider type such as int, values must be checked before narrowing to int8_t.

Value	Status
127	Valid
-128	Valid
0	Valid
128	Invalid
-129	Invalid

This avoids hiding testbench bugs through accidental narrowing conversions.

15. Golden matmul() API

The core API should remain small and architecture-independent.

```
Matrix<std::int32_t>
matmul(
  const Matrix<std::int8_t>& A,
  const Matrix<std::int8_t>& W,
  MatmulStats* stats = nullptr
);
```

Its responsibilities are: validate inputs, allocate/zero the output, perform the mathematical multiplication, optionally collect operation statistics, and return C.

16. Golden Algorithm

```
for i = 0 .. M-1
  for j = 0 .. N-1
    int32 accumulator = 0

    for k = 0 .. K-1
      product = A[i][k] × W[k][j]
      accumulator += product

    C[i][j] = accumulator
```

This is deliberately the classic sequential triple-loop algorithm. It is the independent mathematical reference and must not reproduce the future hardware schedule.

17. Independence Rule

The golden model must not contain hardware-specific concepts. Keeping the reference simple is important because copying an RTL architecture into the reference model can reproduce the same design mistake in both implementations.

Concept	C++ golden model
PE	Not modeled

21. Operation Count Examples

Operation	Multiplications	Additions	MACs	Primitive ops
$1 \times 1 \times 1 \times 1$	1	0	1	1
$2 \times 2 \times 2 \times 2$	8	4	8	12
$3 \times 3 \times 3 \times 3$	27	18	27	45
$4 \times 4 \times 4 \times 4$	64	48	64	112
$8 \times 8 \times 8 \times 8$	512	448	512	960
$16 \times 16 \times 16 \times 16$	4096	3840	4096	7936

22. Rectangular Example

```
A = 3×7  
W = 7×5  
C = 3×5
```

```
Multiplications = 3×7×5 = 105  
Additions = 3×5×(7-1) = 90  
MACs = 105  
Primitive ops = 105 + 90 = 195
```

Rectangular support is part of the current golden-model specification; it is not a tiling feature.

23. Operations Are Not Hardware Cycles

A 4×4 multiplication requires 64 multiplications and 48 additions mathematically, but the future hardware contains 16 PEs and performs work in parallel. Consequently, 112 primitive arithmetic operations does not imply 112 clock cycles.

```
Golden model → WHAT computation is required  
RTL → HOW computation is spatially/temporally performed  
Performance → HOW MANY cycles the hardware actually needs
```

24. Output Initialization

The output matrix C must be initialized to zero before accumulation. A Matrix constructor that value-initializes its `std::vector` storage is sufficient.

```
Matrix<int32_t> C(M, N);  
  
// Each C element starts at 0.  
...  
C(i,j) = accumulator;
```

25. Product and Accumulator Widths

```
std::int32_t acc = 0;  
  
std::int16_t product =  
    static_cast<std::int16_t>(A(i,k)) *  
    static_cast<std::int16_t>(W(k,j));  
  
acc += static_cast<std::int32_t>(product);
```

The exact implementation can be written slightly differently, but the intended widths must remain explicit: signed INT8 operands, signed INT16 product, signed INT32 accumulation.

26. Deterministic Testing

Random testing should use a fixed seed so that the same test can be reproduced exactly.

```
std::mt19937 rng(12345);
```

A reproducible sequence makes software and RTL debugging substantially easier: the same inputs can be regenerated after every failure.

27. Required Test Categories

Test	Purpose
1×1 known result	Basic correctness
2×2 known result	Verify ordinary multiplication
All zeros	Zero propagation
All ones	Accumulation behavior
All −1	Signed arithmetic
127 values	Positive INT8 boundary
−128 values	Negative INT8 boundary
Mixed signs	Signed multiplication
Rectangular matrices	General dimension support
16×16	Maximum supported dimensions

28. Exhaustive Dimension Testing

The software test harness should eventually execute all 4096 combinations of M, K, and N in the range 1..16.

```
for M = 1..16
  for K = 1..16
    for N = 1..16
      create A[M][K]
      create W[K][N]
      calculate C[M][N]
      verify result
```

Dimension coverage and data-value coverage are separate concepts. The harness should combine broad shape coverage with deterministic edge and random values.

29. Test Data Requirements

Test data should include:

- 0
- 1 and −1
- 2 and −2
- +127
- −128
- random positive values
- random negative values
- mixed-sign matrices

All generated INT8 test values must remain within [−128, 127].

30. Verification Philosophy

The central verification comparison is:

```
C_golden ↔ C_RTL
```

Comparison is element by element. A mismatch report should identify enough information to reproduce the failure.

```
ERROR
Matrix: 4x8 × 8x5
Location: C[2][3]
Expected: -1432
Actual: -1408
Difference: 24
```

31. Recommended Directory Structure

```
golden_model/
■■■ matrix.hpp
■■■ matmul.hpp
■■■ matmul.cpp
■■■ main.cpp
```

This simple structure is sufficient for the first implementation. It can later be reorganized into include/, src/, tests/, and a CMake build as the project grows.

32. File Responsibilities

File	Responsibility
matrix.hpp	Generic Matrix<T>, dimensions, storage, indexing, data access
matmul.hpp	matmul() declaration and MatmulStats
matmul.cpp	Validation, multiplication, operation counting
main.cpp	Tests, test data, result printing, pass/fail reporting

33. MatmulStats

```
struct MatmulStats
{
    std::uint64_t multiplications;
    std::uint64_t additions;
    std::uint64_t macs;
};
```

Statistics are software operation counts only. They are useful for sanity checking the algorithm and for later performance/resource discussions, but they are not hardware cycle counts.

34. Current API Boundary

```
A (INT8) ■■
■■■> matmul() ■■■> C (INT32)
W (INT8) ■■■
```

The core API should remain small. Test generation, printing, randomization, and later verification adapters should be layered around the mathematical function rather than embedded into it.

35. Explicitly Excluded From Current Version

The following are intentionally out of scope for Version 1.0:

38. Golden-Model Trust Model

The golden model should be simpler than the design under test. The intended relationship is:

Layer	Role
C++ golden model	Independent mathematical truth/reference
RTL	Hardware implementation under verification
SystemVerilog directed TB	Initial RTL-level verification
UVM scoreboard	Automated expected-vs-actual comparison
Python	Test generation / analysis / secondary checking

Avoid maintaining two equally complex implementations and treating both as independent sources of truth. The reference should remain simple enough to inspect and trust.

39. Frozen Baseline Specification

The current project baseline is:

Implement a generic C++17 software golden model for signed INT8 matrix multiplication, supporting every valid $M \times K \times K \times N$ combination where $1 \leq M, K, N \leq 16$. Inputs are stored as contiguous row-major `std::vector<int8_t>` buffers. Each INT8×INT8 multiplication produces an INT16 product, which is accumulated into an INT32 accumulator and returned as an INT32 output matrix. The implementation uses a simple sequential triple-loop algorithm and remains completely independent of the future systolic hardware architecture. It validates dimensions and input ranges, supports deterministic/random testing, reports multiplication/addition/MAC statistics, and provides element-by-element expected results for later RTL/UVM verification.

Excluded from this baseline: tiling, folding, PE timing, systolic scheduling, AXI, DMA, FPGA resource modeling, quantization, saturation, bias, and activation.

40. Quick Reference

Parameter	Value
Operation	$C = A \times W$
A shape	$M \times K$
W shape	$K \times N$
C shape	$M \times N$
M/K/N range	1..16
Valid shapes	4096
A datatype	signed int8_t
W datatype	signed int8_t
Product	signed int16_t
Accumulator	signed int32_t
Output	signed int32_t
Storage	contiguous row-major <code>std::vector<T></code>
Core algorithm	sequential triple loop
Golden model hardware awareness	None

Parameter	Value
Physical accelerator	4×4 systolic array
Hardware dataflow	weight-stationary
Tiling/folding now	No